Resources:
Raspunsuri intrebari examen:
https://github.com/mariasfiraiala/SO-exam
Notite de curs :
https://drive.google.com/drive/folders/1T08PmJ_fKTA8FLHvhXYw33CBK6D0QPpR
Ce imi place mie:
https://drive.uqu.edu.sa/_/mskhayat/files/MySubjects/2017SS%20Operating%20Systems/Abraham%20Silberschatz-Operating%20System%20Concepts%20(9th,2012_12).pdf

1.Software Stack

What OSs do?

The operating system controls the hardware and coordinates its use among the various application programs for the various users. Like a government, it performs no useful function by itself. It simply provides an environment within which other programs can do useful work.

The fundamental goal of computer systems is to execute user programs and to make solving user problems easier. Computer hardware is constructed toward this goal. Since bare hardware alone is not particularly easy to use, application programs are developed. These programs require certain common operations, such as those controlling the I/O devices. The common functions of controlling and allocating resources are then brought together into one piece of software: the operating system.

A simple viewpoint is that it includes everything a vendor ships when you order “the operating system.”

Software-ul de sistem = SO + bibliotecile standard

Nevoia de mai multe functionalitati are doua consecinte:
-hardware mai puternic, mai performant
-medierea accesului la resursele hardware

Al doilea rol e îndeplinit de sistemul de operare. Sistemele de operare apar pe sistemele de calcul care rulează mai multe aplicații. Fara SO apps ar accesa arbitrar resursele hardware si mai multe malicious apps ar abuza de lipsa de mediere si ar corupe intentionat resursele sistemului

Pentru rolul de mediere: MOD PRIVILEGIAT
Mod privilegiat = kernel mode/space sau supervisor mode/space

SO are astfel rol de furnizor de servicii privilegiate, arbitru al folosirii resurselor și monitor al bunei funcționării a sistemului

Apelul de sistem = un element(functionalitate, metoda, functie) din interfata de sistem expusa de SO apps-urilor pentru actiuni privilegiate

Invocare apel de sistem = schimbarea nivelului de privilegiu

Overhead = orice combinatie de exces a resurselor necesare efectuarii unei operatii
SO e un rău necesar: aduce overhead, dar asigură buna funcționare a sistemului

kernel monolitic = mai multe componente în sistemul de operare -> performanță mai bună, totul în kernel mode, un apel de sistem face mai multe; dar totul este într-un singur loc, suprafață de atac mare

Microkernel = sistem de operare mai mic, suprafață de atac mai mică, performantă mai mică: se fac tranziții mai multe user-kernel pentru a permite componentelor software care rulează în user mode să interacționeze

Componentele critice ale unui sistem formează TCB (Trusted Computing Base). Un sistem axat pe securitate va căuta să minimizeze TCB-ul. TCB e format din hardware, kernel și procesele care rulează ca root.

FCB = structura in care se gasesc metadatele unui fisier(nume, identificator -inode number-, dimesniune, owner, group, permisiuni, timpi de acces

-stat si fstat(in unic) afiseaza metadatele unui fisier

-Un bloc de control al fișierelor (FCB) este o structură de sistem de fișiere în care se menține starea unui fișier deschis. Un FCB este gestionat de sistemul de operare, dar se află în memoria programului care utilizează fișierul, nu în memoria sistemului de operare. Acest lucru permite unui proces să aibă câte fișiere deschise simultan dorește, cu condiția să rezerve suficientă memorie pentru un FCB per fișier.

proces ---> apel de sistem de deschidere ---> structură de fișier deschis ---> structură de fișier pe disc (FCB + date)

-structura de fișier deschis conține: referință către FCB-ul fișierului, permisiuni de deschidere, cursor/pointer de fișier (file pointer/cursor)

-fișierele sunt deschise pentru a fi folosite, se întoarce la nivel low-level un descriptor de fișier

un descriptor este o intrare într-o tabelă de descriptori de fișiere care conține un pointer către o structură de fișier deschis

Un proces este un program care rulează, care servește drept bază pentru toate calculele.

IPC = Inter-Process Communication

Atributele unui proces sunt păstrate în PCB (Process Control Block)

TO BE CONTINUED

Curs 9 - Fire de executie

Un sistem de calcul ofera sprijin pentru realizarea de actiuni ale utilizatorilor.

Actiune = obtinerea unor date de intrare, prelucrarea lor si furnizarea unor date de iesire

I/O intensive: - un editor în general obține date de la tastatură și le scrie pe disc într-un fișier; face prelucrare atunci când face căutări sau înlocuiri

- operează des cu discul, placa de rețea sau alte forme de I/O

CPU intensive - folosesc des procesorul

-un procesor video face encoding/decoding la datele dintr-un fișier

-utilitarul grep obține date din fișier, caută prin ele expresii regulate și scrie rezultatul în alt fișier

<u>Sisteme de calcul multi-core/multi-procesor</u>

Mai multe procesoare = rulam mai multe actiuni diferite fiecare pe cate un procesor

////sau rulam aceeasi actiune in paralel pe mai multe procesoare

Vrem sa rulam aceeasi actiune in paralel pe mai multe procesoare: putem crea mai multe procese de acelasi tip(prin FORK): de ex mai multe procese de tip server-web, de tip procesor video. Avem urmatoarele dezavantaje cand folosim procese:

-pentru ca teoretic noi avem aceeasi actiune, avem nevoie de un set de date comune si este mai dificil de partajat(se poate prin memorie partajata, dar necesita un efort)
-OVERHEAD de memorie pt fiecare proces creat: tabela de pagini noua
-OVERHEAD de schimbare de context: schimbare de tabela de pagini, flush la TLB

Thread-uri(fire de executie)/LWP(Lightweight processes)

Thread = un singur flux secvential de activitati care se executa in cadrul unui proces; o varianta mai simpla a proceselor; sunt de fapt o abstractizare pentru procesor

VAS si descriptorii de I/O apartin procesului si sunt info partajate intre thread-uri

Compenseaza dezavantajele folosirii proceselor astfel:

-se partajeaza usor date: tot VAS al unui proces este partajat intre thread-urile acelui proces
-OVERHEAD de memorie redus pt ca nu se creeaza un spatiu de adrese nou
-schimvbarea de context are penalizare mai mica cand se schimba intre thread-urile aceluiasi proces
-ele simplifica programarea cand e nevoie de DATE PARTAJATE: nu ne trebuie un API pt mem partajata

Dezavantaje:

-partajeaza VAS -> o problema a unui thread, corupe tot
-corupere de date daca un thread foloseste datele altui thread -> problema de sincronizare
-depanare dificila: o problema poate fi cauzata de un thread din memoria altui thread.

Threads vs Processes

Spunem că un proces abstractizează memoria prin spațiul virtual de adrese, I/O prin descriptori de fișiere și procesorul prin thread-uri

Un thread abstractizează doar procesorul

Un proces este pornit dintr-un executabil (exec) sau din procesul curent din punctul curent (fork)

Un thread este abstractizarea cea mai simplă a contextului de execuție; un thread definit simplist printr-un instruction pointer (ce are de executat) și un stack pointer (o stivă, ce a executat până acum)

Schimbarea de context presupune schimbarea unui thread (instruction pointer, stack pointer, celelalte registre) cu alt thread

Un thread este definit de un TCB (Thread Control Block) conținând în general informații de execuție: thread id (TID), stare, stivă, pointer la proces / spațiul de adrese din care face parte, timp de rulare, prioritate

Crearea unui thread -> se indica o functie pe care o va rula ->se aloca o stiva si se incepe rularea acelei functii -> thread-ul devine o entitate planificabila: se poate bloca, ii expira cuanta

Un thread se incheie:

-cand se incheie functia
-cand se incheie procesul curent(un thread al său apelează exit() sau apare o eroare/excepție)
-când se apelează o funcție specifică de închidere a thread-ului (pthread_exit())

Operatia join = este un apel care asteapta incheierea unui thread pentru a recupera datele de iesire

Un thread poate întoarce o valoare la încheierea sa, recuperată cu join; așa cum procesul întoarce un cod de ieșire, recuperat cu wait.

Partajarea spatiului de adrese intre thread-uri

Thread-urile partajeaza spatiul de adrese al procesului. Au memorie partajata

Cream un thread ->TOT spatiul de adrese e partajat

Un thread are stiva proprie, deci variabilele locale sunt proprii fiecarui thread

Dacă are nevoie de variabile globale proprii (ca să nu transmită parametri între funcții) un thread folosește TLS(thread local storage); TLS este o zonă dedicată unde fiecare thread are o referință a acelei variabile.

Implementarea thread-urilor

API-ul de lucru cu thread-uri include:

-functia de creare de thread

-fucntia de asteptare/join a unui thread-functie de identificare a unui thread - TID

-functia de inchidere a unui thread

-primitive de sincronizare

Implementarea poate fi:

-kernel level: thread-urile devin astfel entitati planificabile la nivelul SO. Sunt folosite de SO, au cuanta de executie, stare de ececutie si sunt parte din coada READY. Schimbarea de context necesita interventia nucleului. Avantaje: suport complet multi-procesor, bune pentru acțiuni CPU intensive

-user level: implementare completa in user-space. DAR o operatie blocanta blocheaza tot procesul. SOLUTIE: folosirea de operatii I/O asincrone. Avantaje: nu necesită suport în kernel, timp de activare scurt, performanță ridicată pentru acțiuni I/O intensive

Pentru a reduce timpul de activare se folosesc modele de tipul thread-pool (precum boss-workers sau worker-threads.

Curs 10 - Sincronizare

Atomicitate: operația se execută fără intervenția unui alt proces sau core .

Resursele comune

Faciliteaza interactiunea intre procese si threads dar un thread trebuie sa aiba in vedere ca nu e singurul care acceseaza.

Daca toate threads trebuie sa citeasca -> OK; Daca cel putin un thread trebuie sa scrie ->problema (acces concurent de scriere la date comune, acces de citire când datele nu au fost încă actualizate, sau acces după ce datele au fost curățate/refolosite: race conditions)

Primitive de sincronizare

Pentru mecanisme de acces exclusiv și de atomizare de acțiuni:
-variabile atomice
-mutex(lock/unlock)
-spinlock(lock/unlock)

Pentru ordonarea actiunilor:
-variabile de conditie(wait, notify)
-semafoare(up, down)
-monitoare(enter, leave, wait, notify)
-cozi de asteptare(wait, wake_up)

Functiile de sincronizare au overhead

Legea lui Amdahl: codul serial incetineste programul

Internele primitivelor de sincronizare

Pentru ca primitivele de sincronizare să funcționeze, implementarea lock() și unlock() trebuie să fie atomică

Atomizarea operațiilor se realizează la nivel hardware:
* x86 single core: nu e nevoie
* x86 multi-core: lock pe magistrală: prefixul lock în fața unei operații
* ARM: operații tranzacționale: ldrex, strex; dacă nu reușește eșuează și încerci iar

TOCTOU = time of check to time of use

CAS = compare and swap ->automizarea comparatiei si initializarii ; operatie oferita de procesoare

Spinlock vs Mutex

Spinlock = primitiva de baza pt asigurarea atomicitatii; face busy waiting in asteptarea eliberarii; se bazeaza pe CAS; este util pentru regiuni critice de mici dimensiuni
Mutex = o primitiva mai complexa cu o coada de thread-uri in asteptare; Pnetru protejarea structurilor interne, un mutex are un spinlock; Este util pentru regiuni critice mai mari sau unele in care thread-ul se blocheaza

Performanta transferului. Producator-consumator

Componentele hardware sau software comunica intre ele si isi transmit informatii. Unele produc, altele consuma si una poate fi mai inceata decat cealalta si astfel cea inceata dicteaza comunicarea. Pnetru aceasta problema se foloseste buffering intre cele doua: cea care produce pune elemente în buffer, cealaltă le consumă; dacă e lent consumatorul, producătorul are spațiu de depunere; dacă e lent producătorul, consumatorul poate consuma din ce s-a strâns până atunci.
In mod uzual buffer-ul folosit este un buffer circular: un buffer cu două capete: unul de citire și unul de scriere

Curs 11 - Dispozitive de I/O

Cele 3 resurse ale sistemului de calcul sunt:
-procesorul: folosit pentru prelucrarea de date
-memoria: folosita pentru a retine datele si codul proceselor
-I/O: folosit pentru comunicarea cu exteriorul

Procesele comunica prin interfete de I/O cu:
-utilizatorul: tastatura, mouse
-alte procese:

*de pe acelasi sistem: folosinf fisiere, pipe-uri/fifo-uri, socketi UNIX

*sau de pe alte sisteme: folosind reteaua
-dispozitive dedicate:

*de informare/investigare: senzori

*de control: actuatori

*de stocare: HDD, USB drive, CD-ROM

Comunicarea cu sistemele de I/O este critica si trebuie mediata de SO. De obicei necesita nivelul privilegiat. In general pentru I/O vorbim de niveluri:
-hardware
-SO
-aplicatie

I/O in hardware

procesor->magistrala->controller->dispozitiv

Magistrala reprezinta un numar de linii electrice pe care se transmit date/ comenzi. Datele ajung la controller care este un chip dedicat care are info despre specificul fizic al dispozitivului si il comanda corespunzator. Controller-ul corespunde de zone de memorie(registre). Registrele sunt adresate de procesor printr-o schema dedicata si sunt de 4 tipuri:
-citire
-Scriere
-comanda
-stare

Procesorul foloseste schema de adresare pentru a scrie sau citi informatii din registrul dedicat. Sunt folosite 2 sceheme:
-Port-mapped I/O(isolated I/O): unde un spatiu dedicat de adrese este folosit
-Memory-mapped I/O: un spatiu din adresarea fizica a procesorului esre folosit pentru I/O, adica o parte din adresele fizice ajung in memoria principala, altele ajung in I/O

Memory-mapped este folosita cel mai mult pentru ca simplifica circuitele interne fiindca foloseste o singura schema de adresare

Comunicarea hardware cu dispozitivele I/O

Comunicarea cu dispozitivele de I/O trece prin controller: procesorul scrie în registrele controller-ului, apoi controller-ul le transferă dispozitivului; sau procesorul citește din registrele controller-ului datele primite de la dispozitiv

Procesorul trebuie să știe când poate să citească sau să scrie; riscă să citească date nevalide sau să suprascrie datele care nu au fost trimise; procesorul trebuie să se sincronizeze cu controller-ul. Sincronizarea se poate realiza in 2 feluri:

-polling: procesorul investigheaza CONSTANT starea controller-ului

-intreruperi: procesorul este notificat de controller printr-un semnal electric pe o linie dedicata de disponibilitatea de a primi sau transmite date

In general preferam intreruperile pentru ca nu tin procesorul ocupat, insa nu sunt avantajoase cand vine vorba de un transfer de date mari. Putem tranzita din polling in intrerupere in functie de situatie.

Pentru a minimiza interacțiunea cu procesorul, anumite dispozitive folosesc DMA (Direct Memory Access): un chip dedicat realizează transferul unor blocuri mari de date din controller în memoria principală la comanda procesorului fără ca procesorul să se ocupe de realizarea transferului.

Device drivere

Tratarea intreruperilor si partile care realizeaza transferul de la si catre controller sunt parte din SO, mai exact din DRIVERE.

La device driver ajung întreruperile de la hardware și cererile de comunicare declanșate prin apel de sistem din user space (de exemplu apeluri de forma read() sau write())

Niveluri intermediare pentru prelucrarea I/O

Comenzile si datele tarnsferate de la sau catre user space pot ajunge la device driver direct de la handler-ul unui apel de sistem sau printr-un nivel intermediar la nivelul nucleului. Cest nivel intermediar se numeste generic I/O manager. Este des intalnit la discuti si la placile de retea.

Cresterea performantei lucrului cu I/O

Dispozitivele de I/O sunt de obicei lente iar operatiile sunt adesea blocante si impiedica performantele ridicate. Operatiile read / write sunt considerate sincrone si blocante:

-sincrone: primesc atunci rezultatul operatiei

-blocante: daca nu sunt disponibil epnetru citire sau nu este loc sa se scrie, thread-ul curent se blocheaza, se trece in starea WAITING si se schimba contextul de executie.

O altă variantă este folosirea de operații asincrone (implicit non-blocante): se livrează o cerere către sistemul de operare și apoi operația se întoarce; ulterior se primește o notificare de la sistemul de operare sau se verifică încheierea operației
Operațiile asincrone au avantajul eficienței: nu se blochează thread-ul, se execută altceva în paralel cu operația executată de kernel
Au dezavantajul unui mod de programare complicat: este nevoie de un automat de stări care să fie actualizat pe măsură ce operația evoluează
Modelul de programare asincronă este alternativă la modelul de thread-uri pentru prelucrarea de cereri multiple; de fapt, în biblioteca standard de Linux, API-ul POSIX de operații asincrone e implementat cu thread-uri

Programarea cu thread-uri e mai simplă, dar are un overhead de schimbare de context și probleme de sincronizare; programarea cu operații asincrone este mai complexă dar are un overhead mai redus

Curs 12 - Implementarea sistemelor de fisiere